



Advanced Interaction





Action Class

- User-facing API for emulating complex user gestures
 - Used instead of keyboard and mouse
- Based on a builder design pattern
- **WebDriver Element Commands** cannot handle complex interactions
 - Hover
 - Drag and drop
 - Double click

<https://www.selenium.dev/selenium/docs/api/java/org/openqa/selenium/interactions/Actions.html>



Interactions

Mouse interactions

- click
- click_and_hold
- context_click
- double_click
- drag_and_drop
- drag_and_drop_by
- move_by
- move_to
- release

Keyboard interactions

- key_down
- key_up
- send_keys

Action Builder

```
Actions action = new Actions(driver);  
action. + operations + .perform
```

- Example

```
action.click_and_hold(drag_from).move_to(target).perform()
```



Modifier and Type	Method
Action	<code>build()</code>
Actions	<code>click()</code>
Actions	<code>click(WebElement target)</code>
Actions	<code>clickAndHold()</code>
Actions	<code>clickAndHold(WebElement target)</code>
Actions	<code>contextClick()</code>
Actions	<code>contextClick(WebElement target)</code>
Actions	<code>doubleClick()</code>
Actions	<code>doubleClick(WebElement target)</code>
Actions	<code>dragAndDrop(WebElement source, WebElement target)</code>
Actions	<code>dragAndDropBy(WebElement source, int xOffset, int yOffset)</code>



Example

```
private WebDriver driver;
```

```
@Before
```

```
public void setUp()
```

```
{
```

```
    System.setProperty("webdriver.chrome.driver", "resources/chromedriver.exe");
```

```
    driver = new ChromeDriver();
```

```
    driver.manage().window().maximize();
```

```
    driver.get("https://demoqa.com/menu/");
```

```
}
```

```
@test
```

```
public void testHover()
```

```
{
```

```
    WebElement menu2 = driver.findElement(new By.ByLinkText("Main Item 2"));
```

```
    Actions action = new Actions(driver);
```

```
    action.moveToElement(menu2).perform();
```

```
    WebElement menu_sub = driver.findElement(new By.ByPartialLinkText("SUB SUB LIST"));
```

```
    action.moveToElement(menu_sub).perform();
```

```
    WebElement menu_sub_sub = driver.findElement(new By.ByPartialLinkText("Sub Sub Item 1"));
```

```
    action.moveToElement(menu_sub_sub).click().perform();
```

```
}
```



UNIVERSITAS
GADJAH MADA

Wait Strategy

Why should we wait?

- Element must be present and displayed on the page in order for Selenium to interact with it
 - loaded JavaScript assets often result in changes to the site
 - elements get dynamically added to a page or change visibility based on a click
- Flaky test
 - sometimes the Selenium code executes first before the browser gets into the right state

Selenium Wait Strategy

- Thread/ timeunit sleep (not a best practice)
- Selenium wait
 - Implicit
 - Explicit

<https://www.selenium.dev/documentation/webdriver/ waits/>

Implicit Wait

- Implicit wait is a **global setting** that applies to every element location call for the entire session
- the driver will wait for the duration of the provided value before returning the error.
- as soon as the element is located, the driver will return the element reference and the code will continue executing.
 - larger implicit wait value won't necessarily increase the duration of the session

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(2));
```

Explicit Wait

- loops added to the code that poll the application for a specific condition to evaluate as true before it exits the loop and continues
- specify the exact condition to wait for in each place it is needed

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(5));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.Name("q")));
```

For other conditions, check:

<https://www.selenium.dev/selenium/docs/api/java/org/openqa/selenium/support/ui/ExpectedConditions.html>

Tugas 1 (Action)

<https://the-internet.herokuapp.com/>

1. Hover
 - Hover the first image and assert that the name displayed below is **"name: user1"**
2. key_presses
 - Pressed SHIFT inside the input and assert that the text displayed below is **"You entered:SHIFT"**
3. Drag and drop
 - drag box A to box B and assert that **box B has switch with box A**



Tugas 2 (wait strategies)

Buat kode pengujian untuk studi kasus **test case 1 dan 5** pada halaman web <https://practicetestautomation.com/practice-test-exceptions/>. (test case dijelaskan dalam halaman web).

Untuk tiap test case buatlah kode pengujian sesuai langkah pengujian dan jalankan kode pengujian.

- Exception apa yang muncul? Jelaskan apa yang dimaksud dengan exception tersebut dan mengapa exception tersebut terjadi!
- Lakukan perubahan pada kode anda memanfaatkan `thread.sleep()`, `implicit wait`, dan/atau `explicit wait`. Jalankan pengujian dan observasi perubahan yang terjadi!
- Simpulkan perbedaan dari `thread.sleep()`, `implicit wait`, dan `explicit wait`!
- **Bonus nilai bagi yang mencoba mengerjakan test case 2,3,dan 4**